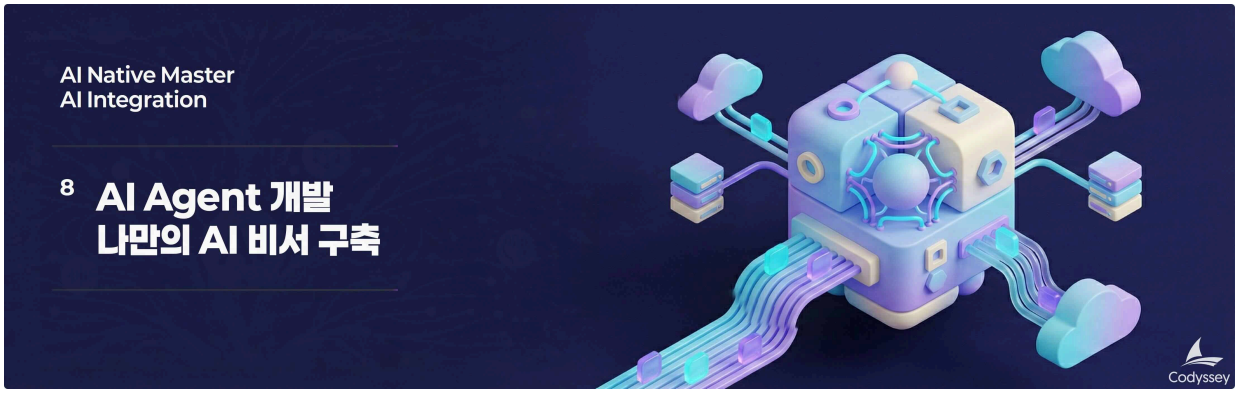
분야 AI 응용 학습 구분 AI 응용 학습시간 80시간 필수/선택 필수

AI Agent 개발: 나만의 AI 비서 구축

문제기술

기술적 설명



1. 미션 소개

일반적인 ChatGPT는 여러분의 데이터를 모릅니다. "이번 달 매출이 어때?"라고 물어도 일반적인 답변만 돌아옵니다.

이 미션에서는 내 데이터를 이해하고 맞춤형 답변을 제공하는 AI 서비스를 만듭니다.

시계열 데이터를 분석하고, 그 결과를 데이터베이스에 저장하고, AI가 이 데이터를 기반으로 대화하는 웹 서비스를 완성합니다.

"내 상황을 아는 AI 비서"를 직접 만들어보세요.

2. 최종 결과물

다음 4가지 기능이 정상 동작하는 애플리케이션 1개를 완성한다.

1. 데이터 기반 AI 채팅
    - 입력/요청: 사용자의 질문(자연어)
    - 출력/화면: 저장된 데이터 요약을 반영한 AI 답변 + 로딩 표시
2. 데이터 관리(CRUD)
    - 입력/요청: (date, value, memo) 형태의 데이터 추가/수정/삭제
    - 출력/화면: 데이터 목록 갱신 및 저장 결과 확인
3. 대화 기록 저장 및 불러오기

- 입력/요청: 대화 저장, 목록 조회, 특정 대화 불러오기
- 출력/화면: 대화 목록과 선택한 대화의 메시지 재표시

4. 배포 및 문서화

- 입력/요청: 배포 URL 접속(프론트/백엔드)
- 출력/화면: 서비스 접속 가능 + Swagger UI 확인 + README로 실행/환경변수 안내

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- 시계열 데이터를 분석하고, 요약 정보를 만들어 서비스에서 활용하는 흐름을 설명할 수 있다.
- FastAPI 프로젝트를 라우터/서비스 등으로 분리해 구성한 기준을 설명할 수 있다.
- Pydantic을 활용해 요청 데이터 검증을 적용한 이유와 방식을 설명할 수 있다.
- Firestore에 데이터를 저장하고 CRUD로 다루는 방법을 설명할 수 있다.
- “데이터 요약을 시스템 프롬프트에 주입하는 방식(컨텍스트 주입)”의 원리를 설명할 수 있다.
- 배포 환경에서 CORS/환경변수/키 관리가 왜 필요한지 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 개발 환경 준비

- Python 3.10 이상, 가상환경(venv) 구성
- 사용 패키지: fastapi, uvicorn, firebase-admin, openai, python-dotenv
- Firebase 프로젝트 생성 및 서비스 계정 키 준비
- OpenAI API 키 준비
- Render, Vercel 계정 준비

2. 데이터 선정 및 분석

- 본인이 관심 있는 시계열 데이터를 선정한다. (예: 주식/환율, 날씨, 판매량, 운동 기록, 학습 시간 등)
- 최소 100개 이상의 데이터 포인트를 준비한다.
- 분석 결과로 “요약 정보”를 만들 수 있어야 한다.

    - 예: 기간, 개수, 기본 통계(평균/최대/최소 등), 최근 추세(증가/감소/유지)

3. FastAPI 프로젝트 구성

- FastAPI 앱 초기화 및 CORS 설정
- 로컬에서 실행 확인 (예: `uvicorn main:app --reload`)
- Swagger UI(`/docs`)에서 API 문서 확인 가능

4. Firebase(Firestore) 연동

- Firestore Database를 사용한다.
- 서비스 계정 키는 코드에 하드코딩하지 않고 환경 변수로 관리한다.
- 컬렉션 구조를 설계한다.
    - data: 분석 데이터 저장
    - conversations: 대화 기록 저장

5. 데이터 API 개발

- 아래 엔드포인트 5개(CRUD 4개 + summary 1개)를 구현한다.
    - POST /api/data : 새 데이터 추가
    - GET /api/data : 데이터 목록 조회
    - PUT /api/data/{id} : 데이터 수정
    - DELETE /api/data/{id} : 데이터 삭제
    - GET /api/data/summary : 데이터 요약(프롬프트 주입용)

6. 대화 기록 API 개발

- 아래 엔드포인트를 구현한다.
    - POST /api/conversations : 대화 저장
    - GET /api/conversations : 대화 목록 조회
    - DELETE /api/conversations/{id} : 대화 삭제
- “대화 불러오기” UX를 위해 아래 중 하나를 만족한다.
    - (A) GET /api/conversations/{id} : 특정 대화의 전체 messages를 조회
    - (B) GET /api/conversations 응답에 messages 포함 여부를 명확히 정의하고 그대로 구현

7. AI 챗봇 API 개발 (컨텍스트 주입)

- POST /api/chat : AI 대화 API를 구현한다.
- 동작 흐름
    - 데이터 요약 조회(/api/data/summary)
    - 요약을 시스템 프롬프트에 삽입
    - GPT API 호출
    - 대화 내용을 conversations에 저장(자동 저장)

8. 백엔드 배포(Render)
    - GitHub에 백엔드 코드 푸시
    - Render에서 Web Service 생성 및 배포
    - 배포 URL에서 Swagger UI(/docs) 확인 가능해야 한다.
    - 무료 티어의 첫 요청 지연(슬립/콜드스타트)을 고려해 사용자 안내 문구 또는 대응을 준비한다.

9. 웹 프론트엔드 개발(바닐라)
    - HTML/CSS/JavaScript로 구현한다. (프레임워크 사용 금지)
    - 필수 화면/기능
        - 채팅 인터페이스: 메시지 입력, 대화 표시, 로딩 표시
        - 데이터 관리: 새 데이터 추가, 목록 표시, 수정/삭제 중 최소 1개 동작이 화면에서 확인 가능
        - 대화 기록: 이전 대화 목록, 대화 불러오기
        - 데이터 요약: 현재 요약 정보 표시(기간/개수/트렌드 등)

10. 프론트엔드 배포(Vercel) 및 문서화
    * Vercel에 배포하고, 환경 변수로 API 서버 URL을 설정한다.
    * README.md에 아래를 포함한다.
    * 서비스 소개(무엇을 해결하는지)
    * 기술 스택
    * 배포 URL(프론트, 백엔드 API, Swagger)
    * 로컬 실행 방법
    * 환경 변수 목록(최소 세트)
    * 제출 스크린샷
    * 데이터 요약이 보이는 채팅 화면 (질문+답변 포함)
    * 데이터 관리 화면 (CRUD 중 1개 동작이 보이게)
    * 대화 기록 화면 (불러오기 동작이 보이게)

5. 보너스 과제 (선택)

- AI 도구 호출(Function Calling) + 멀티채널 연동
    - GPT가 필요 시 데이터 요약/대화 조회 등 내부 기능(API)을 “도구”로 호출하도록 스키마를 정의하고 연결한다.
    - 동일 기능을 MCP Server 또는 GPT Actions 중 1개 방식으로도 연동해(외부 채널/클라이언트) 호출 흐름을 검증한다.
    - README에 “어떤 근거로 어떤 도구를 호출했는지”와 호출 흐름(간단 다이어그램 또는 단계)을 정리한다.
- 인사이트·UX 고도화 (시각화 + 내보내기 + 선택 UI)
    - 요약/통계 정보를 확장해(예: `/api/data/summary` 보강 또는 `/api/data/statistics` 추가) 추가 지표 1개 이상을 제공한다.
    - 프론트에서의 시각화(그래프 1개)로 데이터 흐름(추세/변화)을 보여준다.
    - 데이터 내보내기(CSV 또는 JSON 다운로드) 기능을 제공한다.
    - 다크 모드 토글을 추가한다.

개발환경

6. 개발 환경

- 백엔드는 FastAPI로 구축해야 한다.
- 프론트엔드는 HTML/CSS/JavaScript로 구현한다.
- DB는 Firebase Firestore를 사용해야 한다.
- GPT API를 사용하여 AI 기능을 구현해야 한다.

제약조건

7. 제약 사항

- 배포 고정
    - 백엔드는 Render에 배포해야 한다.

- 프론트엔드는 Vercel에 배포해야 한다.

- 보안 및 운영 기본
  - API 키/서비스 계정 키는 환경 변수로 관리하고 코드에 노출하지 않는다.
  - 입력 값 검증과 최소한의 예외 처리를 적용한다.

- 환경 변수 예시
  - OPENAI_API_KEY
  - FIREBASE_SERVICE_ACCOUNT_JSON (또는 서비스 계정 키 경로)
  - API_BASE_URL (프론트에서 백엔드 주소)
  - ALLOWED_ORIGINS (CORS 허용 도메인 목록)

- 비용/과금 주의
  - 개인 OpenAI 키 사용 시 과금이 발생할 수 있다.
  - 개발/테스트 단계에서 요청 횟수 및 토큰 제한(max_tokens 등)을 두고, 작은 테스트 데이터로 먼저 검증한다.

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구와 디자인은 달라도 된다.

- 데이터 요약 API 응답

```css
{
  "period": "2024-07 ~ 2025-01",
  "count": 180,
  "metrics": {
     "total": 32000000,
   "average": 5330000,
   "max": 8500000,
   "min": 2100000
  },
  "trend": "상승 (월평균 +12%)"
```

```
}
```

- 시스템 프롬프트

```CSS
당신은 데이터 분석 비서입니다.

[사용자 데이터 요약]
- 데이터 기간: {period}
- 총 레코드: {count}개
- 주요 지표: {metrics}
- 최근 트렌드: {trend}

위 데이터를 기반으로 맞춤형 답변을 제공하세요.
```

- AI 대화 예시(예시)

```CSS
사용자: "이번 달 실적이 어때?"
AI: "현재까지 620만원으로, 월평균 대비 16% 높은 수준이에요. 최근 상승 추세가 이어지고 있어요."

사용자: "가장 실적이 좋았던 때는?"
AI: "지난 11월에 850만원으로 최고 실적을 기록했어요."
```